

2장

FastAPI 시작하기

1장에서 FastAPI의 특징을 이해하고 실습 환경을 설정했습니다. 2장에서는 클라이언트와 서버 간에 통신할 때 FastAPI가 제공하는 기능을 이해하고 간단한 API를 만들어 요청을 보내고 응답받는 과정을 실습해 보겠습니다. 실습을 따라 하며 FastAPI의 매끄럽고 직관적인 개발을 경험해 봅시다.

2.1

HTTP 개요

멀리 떨어진 컴퓨터끼리 네트워크상에서 데이터를 주고받으려면 일정한 규칙이 필요합니다. HTTP(HyperText Transfer Protocol)는 이를 위해 만들어진 표준 통신 프로토콜로, 웹에서 클라이언트와 서버 간 정보 교환을 위해 사용합니다. 우리가 웹 브라우저에서 주소를 입력하고 페이지를 불러오는 과정은 모두 HTTP를 통해 이루어집니다.

HTTP는 요청(request)과 응답(response)이라는 구조로 동작하며, 누구나 같은 방식으로 정보를 주고받을 수 있습니다. HTTP 통신을 할 때 클라이언트가 서버로 보내는 메시지를 요청 메시지(request message), 서버가 클라이언트로 보내는 메시지를 응답 메시지(response message)라고 합니다.

2.1.1 요청 메시지

요청 메시지는 요청 라인(request line), 요청 헤더(request headers), 빈 라인(blank line), 요청 본문(request body)의 네 부분으로 구성됩니다. 흔히 로그인이라고 부르는 기능을 요청 메시지로 표현하면 다음과 같습니다.

요청 메시지 예

```
POST /login HTTP/1.1          ——— 요청 라인
Host: www.example.com         ——— 요청 헤더
Content-Type: application/json
Content-Length: 60
                              ——— 빈 라인
{                             ----- 요청 본문
  "username": "Alice",
  "password": "Password123!"
}
```

2.1.2 응답 메시지

응답 메시지는 상태 라인(status line), 응답 헤더(response headers), 빈 라인(blank line), 응답 본문(response body)의 네 부분으로 구성됩니다. 로그인 요청이 성공하면 서버는 다음과 같은 응답 메시지를 보냅니다.

응답 메시지 예

```
HTTP/1.1 200 OK                 --- 상태 라인
Content-Type: application/json --- 응답 헤더
Content-Length: 85
                                --- 빈 라인
{                               --- 응답 본문
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6 ... "
}
```

2.2

경로 다루기

경로(path)는 클라이언트가 서버로 요청을 보내는 주소로, 서버의 기본 주소(http://127.0.0.1:8000) 뒤에 붙는 주소를 말합니다. 예를 들어 http://127.0.0.1:8000/login이라는 주소가 있다면 /login이 경로에 해당하며, 이 경로를 통해 클라이언트의 요청이 서버로 전달됩니다.

2.2.1 기본 사용법

경로의 기본 사용법을 알아보기 전에 데코레이터가 무엇인지 짚고 넘어가겠습니다.

데코레이터(decorator)는 함수를 감싸서 실행 흐름이나 동작을 확장하는 파이썬 문법입니다. FastAPI는 이 데코레이터 문법을 활용해 HTTP 요청과 파이썬 함수를 연결합니다. 즉, FastAPI의 경로 데코레이터 사용 방법을 일반화하면, 다음과 같이 특정 HTTP 메서드와 URL 경로를 함수에 매핑하는 형태로 표현할 수 있습니다.

2.2.2 경로 변수

`/login`은 경로 자체가 고정된 정적 경로(static path)입니다. FastAPI는 경로의 일부를 바꿔 처리할 수 있는 동적 경로(dynamic path)를 지원하는데, 여기서 동적(dynamic)이라는 말은 경로의 일부가 고정된 값이 아니라 요청할 때마다 달라질 수 있다는 뜻입니다. 예를 들어 사용자 아이디에 따라 해당 사용자 정보만 보여 주고 싶다면 경로를 `/users/{user_id}`와 같이 정의합니다.

2.2.3 엔드포인트

지금까지 경로를 이용해 요청을 보내고 응답을 받는 방법, 경로 변수를 이용해 동적 경로를 구성하는 방법을 익혔습니다. API는 이렇게 클라이언트가 정해진 규칙(경로, 메서드, 데이터 형식)에 따라 요청을 보내고, 서버가 이를 처리해 응답하는 약속된 방식을 말합니다.

API는 문맥에 따라 두 가지 의미로 사용됩니다. 넓은 의미의 API는 서비스가 외부에 제공하는 기능 전체를 뜻하며, 좁은 의미의 API는 특정 기능을 수행하는 개별 요청을 뜻합니다. 이 책에서는 지금부터 좁은 의미의 API로 사용합니다.

2.3

쿼리 파라미터 다루기

쿼리 파라미터(query parameters)는 경로 뒤 `?` 다음에 '`key=value`' 형태로 붙어 클라이언트가 선택적으로 전달하는 데이터입니다. 예를 들어 `/items?max_price=1000`과 같이 `/items` 경로 뒤에 `max_price=1000`이라는 쿼리 파라미터를 추가하면, 서버는 이 값을 상품 조회 조건(최대 가격 상한)으로 사용해 가격이 1000 이하인 상품만 조회할 수 있습니다.

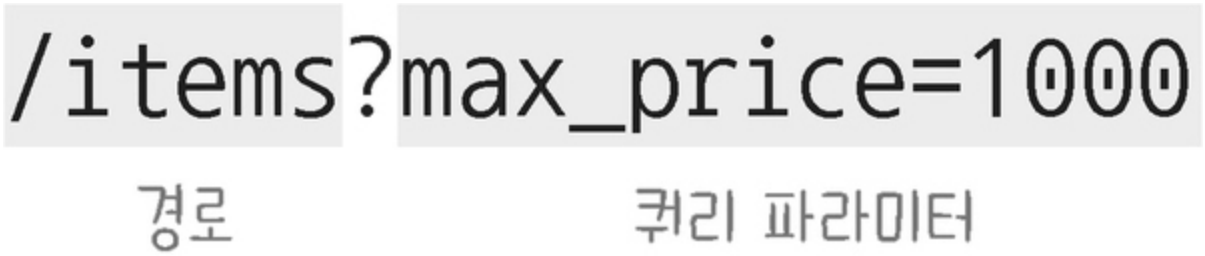


그림 2-4 경로와 쿼리 파라미터

쿼리 파라미터는 `/items?max_price=1000&min_price=100`과 같이 `&` 기호를 사용해 여러 개의 파라미터를 전달할 수 있습니다.

쿼리 파라미터는 반드시 포함할 필요는 없고 특정 검색 키워드를 전달해야 할 때, 필터링이 필요할 때, 페이지네이션을 해야 할 때 주로 사용합니다.

2.3.1 기본 사용법

쿼리 파라미터는 요청 메시지의 요청 라인 안에 들어 있습니다. 다음 요청 메시지에서 진하게 표시된 부분이 쿼리 파라미터입니다.

```
GET /items?max_price=1000 HTTP/1.1
(후략)
```

2.3.2 쿼리 파라미터의 선택적 사용

엔드포인트 함수의 매개변수를 `max_price: int`라고 지정하면 FastAPI는 이 값을 필수로 요구합니다. 따라서 경로에 `?max_price=1000`을 포함하지 않으면 FastAPI가 에러를 발생시킵니다.

하지만 쿼리 파라미터는 대부분 선택적으로 사용되므로 서버에서 항상 이 값을 요구할 필요는 없습니다. 예를 들어 사용자가 상품을 조회할 때 항상 최대 가격을 지정하지 않을 수 있습니다.

> 💡 **[참고] Pydantic v2 호환 설정 (schema_extra ➔ json_schema_extra)**
> FastAPI 최신 버전들은 내부적으로 Pydantic v2를 채택하여 동작합니다.
> v1에서 Swagger 스키마 예제 지정을 위해 쓰였던 `schema_extra`는 v2에서 `json_schema_extra`로 속성명이 격상되었습니다.
> 따라서 모델 구성 시 예시값 데이터는 `model_config = { "json_schema_extra": { "examples": [ ... ] } }` 형태로 선언해주어야 Swagger 문서상에 올바르게 적용됩니다.

2.4

Pydantic 다루기

앞서 살펴본 것처럼 FastAPI는 타입 힌트를 통해 경로 변수와 쿼리 파라미터의 값을 자동으로 검사하며, 이를 위해 내부적으로 `Pydantic`이라는 라이브러리를 사용합니다. Pydantic은 파이썬의 대표적인 데이터 검증 라이브러리로, 입력된 데이터가 올바른 형식인지 확인하는 데 사용합니다.

입력된 데이터를 검증하는 과정은 프로그래밍에서 매우 중요합니다. 잘못된 데이터가 시스템 내부로 들어오는 것을 막아 프로그램의 안정성을 높이고, 보안 취약점을 줄이며, 예측 불가능한 동작을 예방할 수 있기 때문입니다. 또한 데이터 검증을 거치면 사용자에게 명확한 피드백을 제공할 수 있습니다. 예를 들어 숫자를 입력해야 하는 자리에 문자열을 보냈을 경우 '정수가 필요합니다'와 같은 오류 메시지를 반환하여 사용자가 잘못된 입력을 빠르게 수정할 수 있도록 도울 수 있습니다.

> 💡 **[참고] Pydantic v2 호환 설정 (schema_extra ➔ json_schema_extra)**
> FastAPI 최신 버전들은 내부적으로 Pydantic v2를 채택하여 동작합니다.
> v1에서 Swagger 스키마 예제 지정을 위해 쓰였던 `schema_extra`는 v2에서 `json_schema_extra`로 속성명이 격상되었습니다.
> 따라서 모델 구성 시 예시값 데이터는 `model_config = { "json_schema_extra": { "examples": [ ... ] } }` 형태로 선언해주어야 Swagger 문서상에 올바르게 적용됩니다.

2.4.1 기본 사용법

`Pydantic`을 실습하기 위해 툴바의 파이썬 아이콘을 클릭하여 파이썬 콘솔을 엽니다. 파이썬 콘솔(python console)은 코드를 한 줄씩 실행하며 결과를 바로 확인할 수 있는 대화형 환경으로, 간단하게 파이썬 코드를 실습하고 테스트하는 데 편리합니다.



그림 2-7 파이썬 콘솔 열기

`Pydantic`으로 데이터를 검증하려면 `BaseModel`을 상속받아 `Pydantic` 모델을 정의해야 합니다. 이렇게 정의한 모델을 활용하면 요청 데이터가 지정된 타입과 구조를 따르는지 자동으로 검사하고 잘못된 값이 들어오면 오류를 발생시킵니다. 이 과정을 파이썬 콘솔에서 실습해 보겠습니다.

2.4.2 정상 값 검증

`Pydantic` 모델을 활용하여 데이터가 올바른지 확인해 보겠습니다. 앞에서 만든 `Item` 모델에 값을 넣어 `Item` 객체를 생성하면 `Pydantic`이 각 필드의 타입과 유효성을 자동으로 검사해 줍니다.

2.4.3 비정상 값 검증

이번에는 `Item` 모델에 비정상 값을 전달했을 때 `Pydantic`이 어떻게 검증하고 오류를 발생시키는지 살펴보겠습니다.

CONSOLE

```
>>> data = {"name": 123, "price": "abc"} --- ❶
>>> new_item = Item(**data) --- ❷
```

❶ `data` 딕셔너리에 새 `Item` 객체를 만들 때 필요한 값을 넣습니다. 검증을 통과하지 못하도록 `name` 필드에 문자열이 아닌 숫자(`123`)를, `price` 필드에 정수가 아닌 문자열(`"abc"`)을 넣습니다.

❷ 이번에는 검증할 값이 올바르지 않기 때문에 `Item(**data)`를 실행하면 `Pydantic`이 타입 오류를 감지하고 `ValidationError`를 발생시킵니다.

2.5

요청 본문 다루기

`Pydantic`이 가장 직접적으로 사용되는 부분은 요청 본문입니다. 요청 본문은 클라이언트가 서버로 요청을 보낼 때 함께 전송하는 데이터입니다. FastAPI가 어떻게 요청 본문을 선언하고 처리하는지 살펴보겠습니다.

2.5.1 기본 사용법

새 아이템을 등록하는 요청을 보낸다고 가정해 봅시다. 다음과 같이 제품명(`name`), 정가(`price`), 재고(`in_stock`) 정보가 요청 본문에 JSON 형태로 담겨 서버로 전달됩니다.

요청 메시지

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/json
Content-Length: 60
```

요청 본문

```
{
  "name": "pencil",
  "price": 1000,
  "in_stock": true
}
```

2.5.2 Swagger UI 화면 구성

잠시 Swagger UI에 대해 짚고 넘어가겠습니다. Swagger UI는 FastAPI가 자동으로 생성하는 API 문서 및 테스트 도구입니다. 각 API 엔드포인트에 대해 요청과 응답을 시각적으로 보여 주고 직접 테스트할 수 있습니다.

Swagger UI는 총 8개 영역으로 구성됩니다. 화면이 길기 때문에 순서대로 구분해 설명합니다.

2.5.3 경로 변수, 쿼리 파라미터, 요청 본문 혼합 사용

엔드포인트 함수의 매개변수에 경로 변수, 쿼리 파라미터, 요청 본문을 모두 사용할 수 있습니다.

2.6

응답 본문 다루기

이번에는 서버가 HTTP 응답을 어떻게 반환하는지 알아보겠습니다.

2.6.1 기본 사용법

지금까지는 엔드포인트 함수에서 파이썬의 딕셔너리를 그대로 반환했습니다. 즉, 함수 안에서 `{"message": "...", "value": ...}`와 같은 딕셔너리를 만들고 이를 반환하면 FastAPI가 이 값을 자동으로 JSON 형식으로 변환해 클라이언트에 전달했습니다.

서버가 응답을 보낼 때 딕셔너리를 사용하는 방법 외에도 `Pydantic` 모델을 활용하는 방법도 있습니다. `Pydantic` 모델을 사용하면 반환 데이터의 구조와 타입을 명확하게 정의하여 잘못된 데이터가 클라이언트에 전달되는 것을 방지할 수 있습니다. 또한 모델 정보가 Swagger UI에 반영되므로 API 문서 작성과 테스트가 편합니다.

응답 본문에 `Pydantic` 모델을 활용하려면 엔드포인트 함수의 반환값을 `Pydantic` 모델로 된 객체로 지정하고, 데코레이터에서 `response_model` 옵션으로 해당 모델을 지정하면 됩니다. main.py의 `# 새 아이템 등록` 코드를 다음과 같이 변경하고 저장합니다.

2.6.2 상태 코드

상태 코드(status code)는 클라이언트의 요청에 대해 서버가 어떤 결과를 반환했는지를 숫자와 메시지로 나타내는 표준 규약입니다. 예를 들어 `200 OK`는 요청이 성공적으로 처리되었음을 나타내고, `404 Not Found`는 요청한 데이터를 찾을 수 없음을 나타냅니다.

마무리

1 HTTP

① 클라이언트와 서버 간 데이터를 주고받는 표준 통신 프로토콜입니다.